

HR Interview Assistant

Retrieval-Augmented Generation (RAG) System Architecture & Prompts

Overview: This document outlines the conversational flow, system prompts, and RAG mechanics for an AI-driven HR screening bot. The bot is designed to greet candidates, identify their professional domain, retrieve domain-specific technical questions from a vector database, and conduct a preliminary interview.

1. System Architecture & Flow

To create a dynamic HR bot that asks relevant questions based on the user's domain, the conversational pipeline must be split into two distinct phases:

- **Phase 1: Discovery & Routing** - The LLM acts as a friendly recruiter, greets the candidate, and asks which domain or position they are applying for.
- **Phase 2: RAG-Augmented Interviewing** - Once the domain is identified (e.g., "Software Development", "Marketing", "Finance"), the backend queries the Vector Database for interview guidelines, technical questions, and evaluation criteria specific to that domain. This retrieved context is injected into the LLM's prompt.

2. Phase 1: Discovery Prompt

Use this System Prompt when the conversation first begins. Its sole purpose is to be welcoming and to accurately extract the candidate's domain.

System Role:

You are an enthusiastic and professional Human Resources Screening Assistant.

Your current goal is to welcome the candidate and determine the professional domain or specific role they are applying for.

Instructions:

1. Greet the candidate warmly.
2. Ask them which domain, department, or specific role they are interviewing for today.
3. Keep your response brief and strictly focused on gathering this information.
4. Do NOT start asking interview questions yet.

Example Output:

"Hello! Welcome to our initial screening process. I'm the AI HR Assistant. To ensure I ask you the most relevant questions, could you please tell me which domain or role you are applying for today?"

3. Phase 2: RAG-Augmented Interview Prompt

Once the user replies with their domain (e.g., "I am applying for a Software Developer role focusing on Python and Angular"), your backend should perform a similarity search in your vector database using their response.

Inject the retrieved HR guidelines and questions into the `{retrieved_context}` variable in the prompt below:

System Role:

You are an expert HR Technical Interviewer. You are conducting an initial screening for a candidate in the {user_domain} domain.

Retrieved HR Guidelines & Approved Questions:

{retrieved_context}

Instructions:

1. Acknowledge the candidate's chosen domain briefly and professionally.
2. Conduct the interview by asking ONE question at a time from the retrieved context provided above.
3. Wait for the candidate's response before asking the next question.
4. Evaluate the candidate's response silently against the evaluation criteria in the context. If their answer is too brief, politely ask them to elaborate or provide an example.
5. Do NOT make up your own technical questions. Strictly use the questions provided in the context.
6. Maintain a supportive, encouraging, yet professional HR tone.

Constraint:

Never ask all the questions at once. Always engage in a back-and-forth dialogue.

4. Vector Database Schema (Knowledge Base)

To make the RAG system work, your vector store (Pinecone, ChromaDB, pgvector, etc.) should contain documents structured like the examples below. When a user states their domain, the relevant chunks are fetched.

Domain Chunk (Metadata)	Document Content (Text to Embed)
Software Engineering Tags: Python, Angular, Java	Role: Full-Stack Developer. Q1: "Can you describe a time when you had to optimize the performance of a Python application or an Angular frontend?" (Look for understanding of async processes, change detection, or lazy loading). Q2: "How do you ensure data integrity when passing information between a Java backend and a web client?"
Marketing Tags: SEO, Content, Campaigns	Role: Digital Marketing Specialist. Q1: "Walk me through your process for planning an SEO-driven content campaign." Q2: "How do you measure the ROI of a digital campaign, and what metrics are most important to you?"

5. Backend Implementation Logic (Python/LangChain)

Here is a conceptual flow for the backend orchestration:

```
# 1. State Management
# Keep track of conversation state (e.g., Phase 1 or Phase 2)

if conversation_state == "AWAITING_DOMAIN":
    # User replies: "I do software development"
    domain_query = user_input

    # 2. Retrieve relevant HR questions from Vector DB
    docs = vector_store.similarity_search(domain_query, k=2)
    context = "
".join([doc.page_content for doc in docs])

    # 3. Update State and System Prompt
    conversation_state = "INTERVIEWING"
    current_prompt = Phase2_Prompt.format(
        user_domain=domain_query,
        retrieved_context=context
    )

    # 4. Generate next reply (First interview question)
    response = llm.invoke(current_prompt)
    return response
```